CS401 Intelligent Robotics Lab 03 Report

Date: 2026-03-23

Remote host	ubuntu@192.168.44.131
Operating system	Ubuntu 20.04.6 LTS
ROS version	ROS Noetic 1.17.4
Workspace	~/catkin_ws
Submitted package	learning_tf

1. Objectives

This lab studies ROS tf/tf2, frame broadcasting and listening, visualization tools, and sensor data transformation between robot frames. The work in this report is based on the LAB3 materials in the local repository and the completed remote implementation.

The required tasks were to reproduce the tf2 turtlesim tutorial, implement a custom learning_tf package, publish the base_link to base_laser transform, and convert laser data from the base_laser frame into the base_link frame.

2. Implementation Summary

A ROS package named learning_tf was created in ~/catkin_ws/src. It contains tutorial nodes for the turtlesim tf2 demo and custom nodes for the LAB3 laser transformation task.

The package includes turtle_tf2_broadcaster.py, turtle_tf2_listener.py, fixed_tf2_broadcaster.py, dynamic_tf2_broadcaster.py, turtle_tf2_message_broadcaster.py, turtle_circle_driver.py, rosbot_tf_broadcaster.py, robot_tf_listener.py, test_scan_publisher.py, and the corresponding launch files.

In ROS Noetic Python, laser_geometry does not directly expose transformLaserScanToPointCloud(...). Therefore the implementation uses LaserProjection.projectLaser(...) together with tf2_sensor_msgs.do_transform_cloud(...). This preserves the intended LAB3 behavior.

3. Package Structure

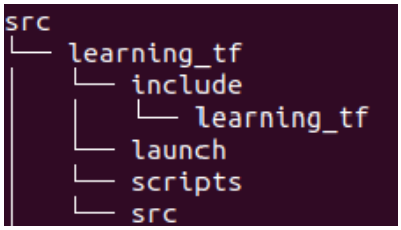

Figure 1. Local package structure of learning_tf.

4. Turtlesim tf2 Tutorial

The turtlesim demo was launched with learning_tf_py.launch. turtle1 published its pose to tf2, and turtle2 listened to the transform and followed the target frame. The demo was also extended with an additional fixed frame named carrot1.

```
cd ~/catkin_ws
catkin_make
```

```
source /opt/ros/noetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch learning_tf learning_tf_py.launch
rosrun turtlesim turtle_teleop_key
rosrun tf view_frames
rosrun rqt_tf_tree rqt_tf_tree
rosrun tf tf_echo turtle1 turtle2
```

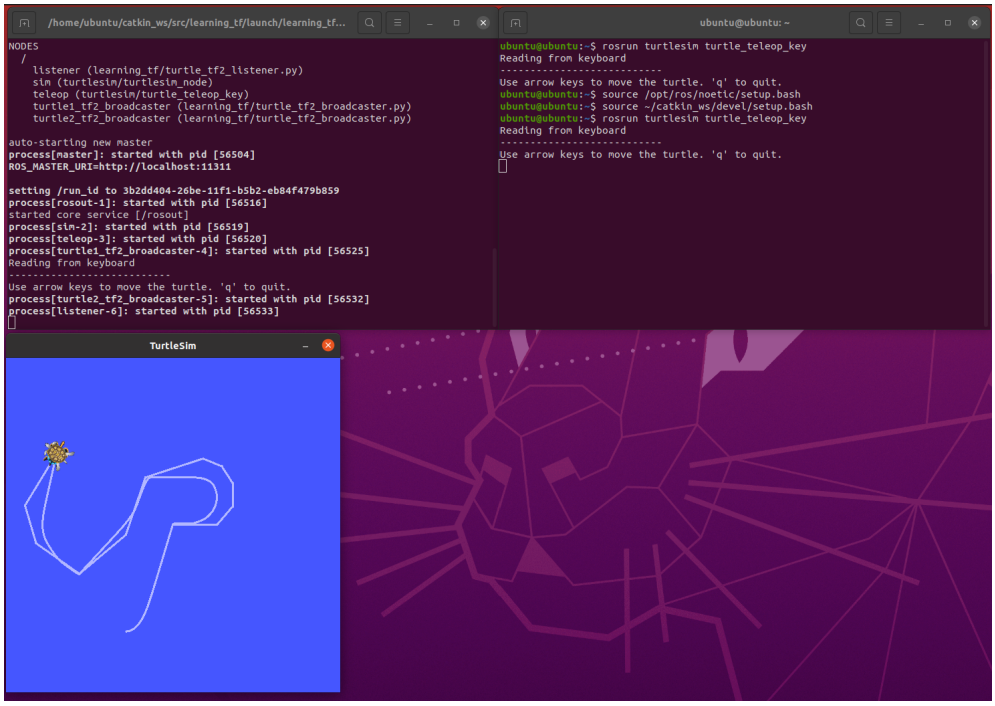

Figure 2. tf2 turtlesim demo with teleoperation and active ROS nodes.

During verification, the transform query between turtle1 and turtle2 returned stable results. One sampled output was Translation [-0.682, 0.933, 0.000] with a valid quaternion rotation. Pose sampling also confirmed that both turtles changed state, which indicates that turtle2 was following the target frame rather than remaining static.

```
/listener
/rosout
/turtle1_auto_driver
/turtle1_tf2_broadcaster
/turtle2_tf2_broadcaster
/turtlesim

turtle1 (6.27, 5.26, -2.869) -> (5.439, 3.633, -1.23)
turtle2 (7.161, 4.537, 2.283) -> (6.104, 4.578, -2.373)
```

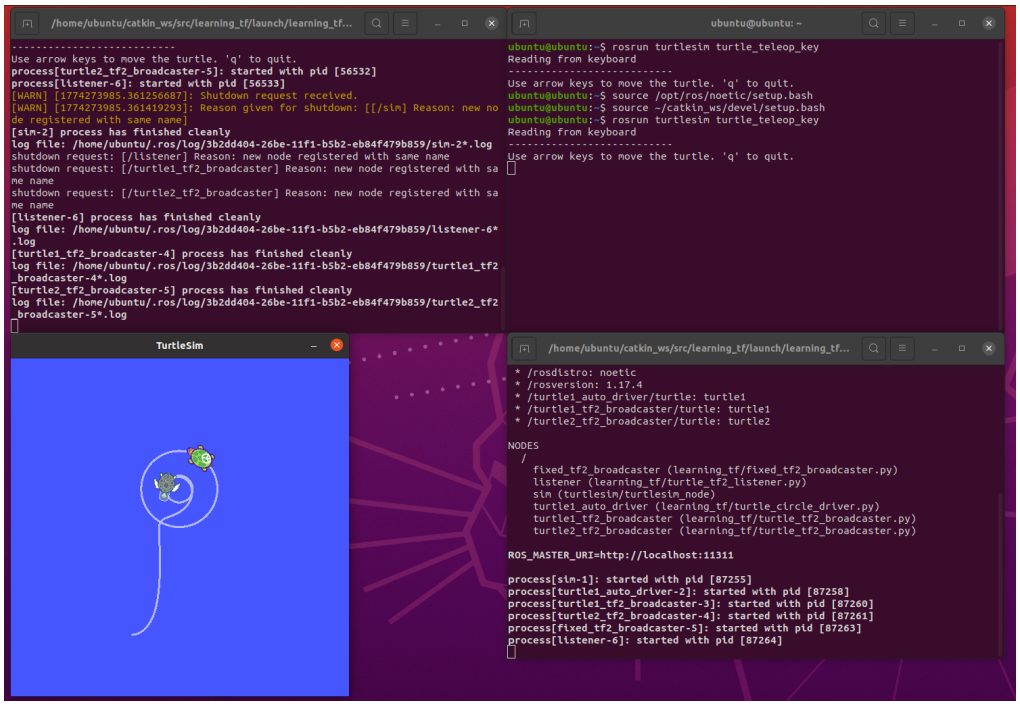

Figure 3. Extended tf2 demo with the fixed extra frame carrot1.

5. Laser Transformation Task

The LAB3 PDF specifies the static transform from base_link to base_laser as x=0.1 m, y=0.0 m, z=0.2 m. The custom node rosbot_tf_broadcaster.py publishes this static transform, and robot_tf_listener.py projects LaserScan to PointCloud2 and transforms the cloud into base_link.

```
roslaunch learning_tf rosbot_tf.launch
rosrun tf tf_echo base_link base_laser
rostopic echo -n 1 /scan_in_base_link/header
rostopic hz /scan_in_base_link
```

The static transform query returned Translation [0.100, 0.000, 0.200] and identity rotation, which matches the LAB3 geometry. The transformed point cloud header used frame_id = base_link, and the topic was published at approximately 5 Hz.

```
At time 0.000
- Translation: [0.100, 0.000, 0.200]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]

seq: 1
frame_id: "base_link"

average rate: 4.994
average rate: 5.004
average rate: 5.000
```

A sampled transformed cloud contained five points because the test launch includes a minimal test scan publisher for deterministic verification. The sampled output was [[0.541, -0.089, 0.2], [0.498, -0.04, 0.2], [0.4, 0.0, 0.2], [0.498, 0.04, 0.2], [0.541, 0.089, 0.2]]. The center point (0.4, 0.0, 0.2) is consistent with the expected 0.1 m frame offset added to a 0.3 m forward scan point.

GUI verification in RViz also confirmed that the transformed cloud was displayed in the base_link frame while tf_echo continuously reported the expected static transform and rostopic hz showed a stable publication rate.

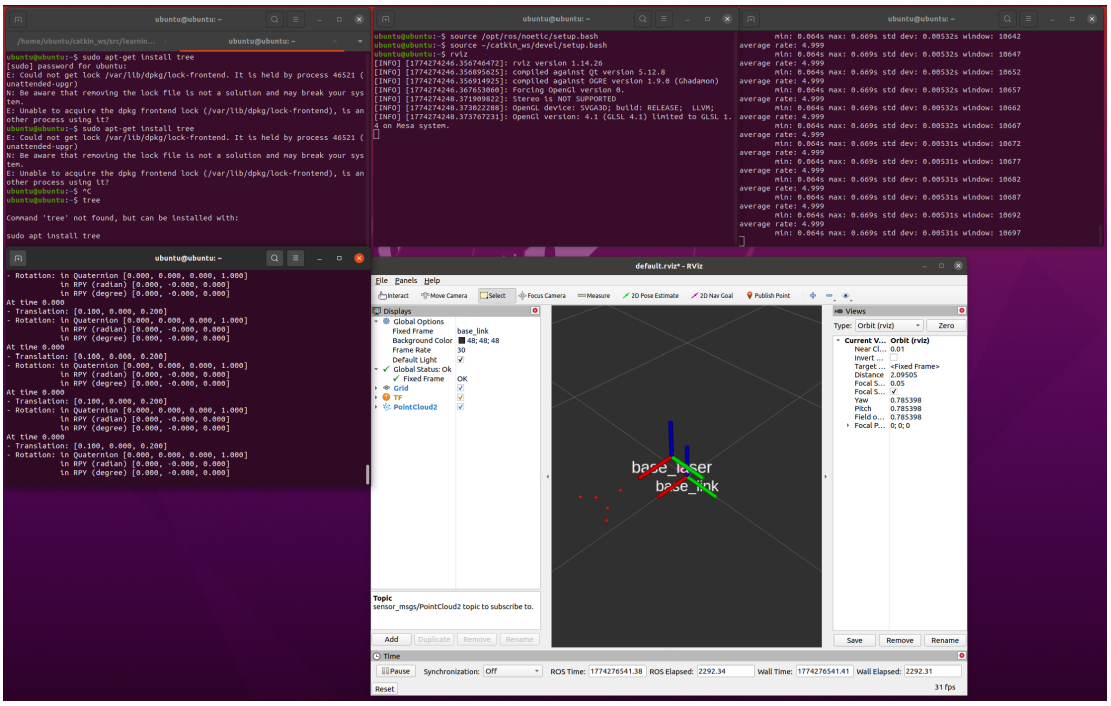

Figure 4. RViz and command-line verification for the base_link to base_laser transform.

6. Visualization Tools

The experiment used tf_echo, rostopic echo, rostopic hz, rqt_tf_tree, and RViz. For the laser transformation demo, RViz was configured with Fixed Frame = base_link and the PointCloud2 topic /scan_in_base_link. This made it possible to verify that the transformed data was displayed in the robot base frame.

7. Conclusion

All required LAB3 tasks were completed. The tf2 turtlesim tutorial ran correctly, the extended frame example worked, the learning_tf package built successfully in catkin_ws, and the laser data was transformed from base_laser into base_link as required. The final deliverables were organized for direct submission.

8. Submission Contents

```
LAB3_report.pdf
learning_tf.zip
submission_manifest.txt
```